

Bases tecnológicas para el servicio

MATERIA: PROGRAMACIÓN EN JAVA

ACTIVIDAD: REPORTE – Estudio de
Caso: Tienda Departamental “Dreams”

ALUMNO: CESAR CEDILLO
RODRÍGUEZ

GRUPO: 8

DOCENTE: JUAN MANUEL
TOLENTINO MORALES

1. Identificación de Clases y Objetos

De acuerdo con las necesidades descritas para la tienda departamental “Dreams”, se identificaron las clases Producto, Stock, Cliente, Venta, Entidad (clase abstracta) y Calculable (interfaz). Estas clases representan los elementos fundamentales para gestionar productos, clientes y ventas dentro del sistema.

2. Definición de atributos y métodos

A continuación, se describen los atributos y métodos principales de cada clase.

2.1 Producto

Atributo	Tipo
nombreProd	String

precioProd double

categoriaProd String

stock Stock

Métodos principales:

getPrecioProd()

getStock()

2.2 Stock

Atributo	Tipo
----------	------

cantidad	int
----------	-----

Métodos principales:

getCantidad()

reducirStock(int n)

2.3 Cliente

Atributo	Tipo
----------	------

nombreCliente	String
---------------	--------

correoCliente	String
---------------	--------

Métodos principales:

getCorreoCliente()

2.4 Venta

Atributo	Tipo
----------	------

producto	Producto
----------	----------

cliente	Cliente
---------	---------

cantidadVenta	int
---------------	-----

Métodos principales:

calcularTotal() ← de la interfaz Calculable

realizarVenta()

2.5 Entidad (abstracta)

Atributo	Tipo
----------	------

nombre	String
--------	--------

Métodos principales:

getNombre()

2.6 Calculable (interfaz)

Método requerido:

calcularTotal()

3. Relaciones entre clases

Las clases mantienen relaciones adecuadas con el problema. Producto contiene un Stock mediante composición. Venta se asocia tanto a Cliente como a Producto, representando una transacción. Producto y Cliente heredan de Entidad, compartiendo el atributo nombre. Además, Venta implementa la interfaz Calculable, asegurando que pueda calcular el monto total de la operación.

4. Reflexión

4.1 ¿Cómo se relacionan las clases entre sí?

- Las clases se relacionan siguiendo un modelo lógico de negocio:
- Las ventas dependen de la existencia de productos y clientes.
- Cada producto está vinculado con su propio stock.
- Cliente y Producto heredan atributos comunes desde la clase abstracta Entidad.
- Venta implementa el comportamiento de cálculo de total mediante la interfaz Calculable.

Estas relaciones permiten organizar el sistema de forma modular y coherente con las operaciones reales de una tienda departamental.

4.2 ¿Qué beneficios aporta la POO en el diseño de este programa?

La POO permite:

Encapsulación del estado del inventario y atributos del cliente. Esto se logra con el modificador de acceso `private` que solo permite acceso dentro de la misma clase.

Reutilización mediante herencia (Entidad). Se generó esta clase con la finalidad de hacer uso de una clase abstracta, con lo que se consigue proteger el nombre del cliente y el del producto.

Flexibilidad gracias al uso de la interfaz `Calculable` que tiene un método abstracto, que las clases que implementan la interfaz pueden definir como mejor sea el caso.

Organización modular, lo que facilita futuras ampliaciones como múltiples productos por venta, facturación, historial, etc.

Mantenibilidad, permitiendo que los cambios en una clase no afecten a las demás.

4.3 ¿Cómo se podría garantizar la precisión en el cálculo de las ventas?

- Usar tipos adecuados (`double` o `BigDecimal` si se requiere precisión absoluta).
- Encapsular la lógica del cálculo en un único método (`calcularTotal()`), evitando duplicación.
- Validar que el stock sea suficiente antes de procesar una venta.
- Realizar pruebas para verificar diferentes escenarios de venta.

4.4 ¿Qué mecanismos de la POO permiten reutilizar atributos y métodos?

- Herencia (por ejemplo, Entidad como clase base).
- Interfaces para compartir comportamientos comunes.
- Clases de apoyo como `Stock`, que pueden ser reutilizadas en otros sistemas.
- Composición, que permite construir clases complejas a partir de clases más simples.

4.5 ¿Qué patrón de diseño sería adecuado para gestionar las ventas?

Aún no vemos el tema

4.6 ¿Qué hacer si un cliente intenta comprar sin stock suficiente?

- Se pueden aplicar diferentes estrategias:
- Mostrar un mensaje de error (“Stock insuficiente”).
- Rechazar la operación y no modificar inventario.
- Registrar un intento fallido para análisis.

En este caso, la clase `Venta` evita realizar la venta y notifica al usuario.

5. Conclusión

El diseño del sistema de tienda departamental permitió aplicar principios clave de la Programación Orientada a Objetos, organizando las clases de forma coherente y eficiente. Las relaciones establecidas, el uso de herencia, interfaces y composición facilitan la expansión futura del programa y mejoran su mantenibilidad. Este enfoque proporciona una estructura clara y modular capaz de adaptarse a necesidades crecientes del negocio.

Diagrama UML de las Clases de la tienda

